

Proyecto Final: API Backend de Gestión de Datos y Optimización

Objetivo: Desarrollar una API robusta que implemente lógica relacional, optimización de consultas y gestión de concurrencia, demostrando un flujo de trabajo profesional mediante el uso de control de versiones.

1. Consideraciones y Entrega

- **Repositorio en GitHub:** El proyecto debe estar alojado en un repositorio de GitHub.
- **Participación Individual (Commits):** Al ser un trabajo grupal, se evaluará el historial de contribuciones. **Se tomará en cuenta el porcentaje y la calidad de los commits de cada participante** para validar su aporte al desarrollo del código y las consultas SQL.
- **Sin ORM:** Todas las interacciones con la base de datos deben realizarse mediante **SQL puro**.
- **Entregables:** Enlace al repositorio que debe contener el código del backend, el script `.sql` para recrear la base de datos y un **README** con las instrucciones de configuración.

2. Rúbrica de Evaluación (Total: 20 puntos)

Entregable	Criterio	Descripción Técnica	Puntaje
Desarrollado en clases	Repositorio y Colaboración (GitHub)	El proyecto está correctamente versionado. Se evidencia una participación equitativa de los miembros del grupo a través de los commits.	2.5
Desarrollado en clases	Diseño de Base de Datos (DDL)	Implementación de 12-15 entidades. Uso correcto de tipos de datos, claves primarias y foráneas.	3.0



Desarrollado en clases	Integridad y Constraints	Uso de CHECK , UNIQUE , NOT NULL y gestión de integridad referencial (CASCADE , SET NULL).	2.0
Entregable 1	Operaciones CRUD Complejas	Endpoints que manejen lógica de 3 o más tablas relacionadas (ej. inserciones en cascada o actualizaciones vinculadas).	4.0
Entregable 1	Reportes y Exportación	Un endpoint que realice agregaciones complejas (GROUP BY , HAVING) y permita exportar el resultado directamente a un archivo CSV/Excel.	2.0
Entregable 2	Optimización (Índices)	Creación de índices justificados con evidencia del plan de ejecución (EXPLAIN) antes y después de la mejora.	3.0
Entregable 2	Gestión de Transacciones	Implementación de un proceso crítico que requiera ACID (ej. una venta o transferencia) usando BEGIN , COMMIT/ROLLBACK .	1.0
	Caso de Uso NoSQL / Híbrido	Documentar o implementar un módulo con tipos de datos semi-estructurados (ej. JSONB) justificando su ventaja sobre el modelo relacional.	2.5
	TOTAL		20.0

